

RelayCaching: Accelerating LLM Collaboration via Decoding KV Cache Reuse

Yingsheng Geng¹ Yuchong Gao² Weihong Wu³ Guyue Liu⁴ Jiang Liu¹

Abstract

The increasing complexity of AI tasks has shifted the paradigm from monolithic models toward multi-agent large language model (LLM) systems. However, these collaborative architectures introduce a critical bottleneck: redundant prefill computation for shared content generated by previous agents, which significantly increases KV cache memory usage and time-to-first-token (TTFT). While various KV cache methods have been proposed to mitigate prefill redundancy, they either fail to maintain accuracy on agent-generated outputs or exhibit low reuse rates due to rigid constraints. We present RelayCaching, a training-free inference method that directly reuses decoding phase KV caches from previous agents in subsequent prefill phases. Our key insight is that KV caches for identical content are highly consistent across phases, while prefix-induced deviations are sparse and localized within a limited range of layers and token positions. By selectively recomputing KV caches at these positions, RelayCaching preserves model accuracy with minimal overhead, yielding a superior accuracy–efficiency trade-off over existing methods. Experiments on diverse collaborative LLM tasks spanning mathematical reasoning, general knowledge, and code generation demonstrate that RelayCaching achieves over 80% KV cache reuse, reduces TTFT by up to $4.7\times$ compared to the standard pipeline, all with negligible accuracy degradation.

1. Introduction

Multi-agent systems (MAS) have emerged as a dominant paradigm for deploying LLMs on complex tasks, orchestrat-

ing specialized agents that collaborate through structured communication (Wu et al., 2024; Hong et al., 2023; Li et al., 2023). Recent frameworks scale MAS to increasingly sophisticated workflows—from multi-turn software development pipelines (Qian et al., 2024) to scientific discovery conducted by teams of coordinated agents (Su et al., 2025)—thereby amplifying both capability and computational demands. In these pipelines, agents exchange intermediate outputs such as reasoning chains, API or code artifacts, and summarized context, enabling task decomposition and iterative refinement (Wu et al., 2024; Hong et al., 2023). As illustrated in Figure 1, consider a development workflow: an Architect agent designs an API specification, a Developer implements it, and a Reviewer evaluates the result. Each upstream output is embedded into downstream prompts, causing cascading redundant prefill.

However, this output-to-input data flow suffers from *prefix variation*: the same tokens appear with different preceding contexts between decoding and prefill phases. This mismatch introduces severe *cascading context redundancy*, as serving systems must treat previously generated text as entirely new input and recompute KV caches from scratch during prefill. In a pipeline with M interaction turns, each agent must reprocess the accumulated history from all preceding interactions, causing cumulative prefill cost quadratically with interaction turns ($O(M^2)$) (Ye et al., 2025). This redundant prefill computation for shared content becomes a critical bottleneck in multi-agent pipelines, significantly increasing KV-cache memory usage and TTFT.

Existing prefill redundancy optimization methods based on *prefix caching* and *pre-computed caching* are inadequate for the dynamic nature of MAS. *Prefix caching* (Kwon et al., 2023; Zheng et al., 2024) relies on strict positional alignment and fails in MAS, where reused content frequently appears at non-prefix positions. *Pre-computed caching* (Gim et al., 2024; Yao et al., 2025) enables non-prefix reuse but assumes static, offline content (e.g., documents in RAG), limiting applicability in MAS where outputs are generated on the fly. Consequently, current systems often fall back to expensive full prefilling in MAS settings (Figure 1(a,b)).

This limitation motivates a fundamental question: *can we directly reuse decoding KV caches to bypass redundant prefill computation despite prefix variation?* To answer this, we

¹Beijing University of Posts and Telecommunications, Beijing, China ²Tsinghua University, Beijing, China ³University of Electronic Science and Technology of China, Chengdu, China ⁴Peking University, Beijing, China. Correspondence to: Weihong Wu <wuweihong@uestc.edu.cn>, Guyue Liu <guyue@pku.edu.cn>.

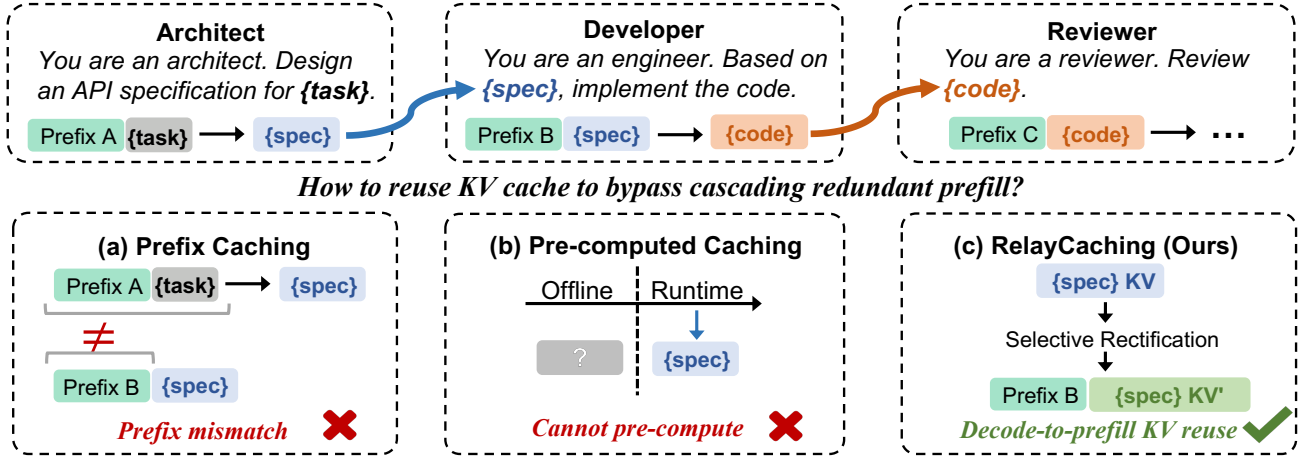


Figure 1. Overview of KV cache reuse in multi-agent LLM collaboration. **Top:** A typical workflow where an Architect generates $\{spec\}$, a Developer produces $\{code\}$ based on it, and a Reviewer evaluates the result, so each agent’s output becomes the next agent’s input, causing cascading redundant prefill. **Bottom:** Comparison of caching strategies. (a) Prefix Caching fails due to prefix mismatch; (b) Pre-computed Caching cannot handle dynamic content; (c) RelayCaching enables decode-to-prefill KV reuse via selective rectification.

conduct a systematic empirical study comparing decoding KV caches with their full-prefill counterparts. Our analysis reveals three key findings that establish the feasibility of direct reuse with targeted rectification: **(1) High macro-level alignment.** Despite prefix variation, decoding KV caches maintain high alignment with full-prefill counterparts, with value cosine similarity emerging as the primary deviation indicator. **(2) U-shaped layer-wise similarity profile.** Middle layers show the largest deviations and dominate subsequent generation quality, while shallow and deep layers remain relatively stable. **(3) Sparse and correlated token-wise deviations.** Token-wise deviations are sparse and exhibit strong inter-layer correlation.

Building on these observations, we propose *RelayCaching*, a training-free method that directly reuses decoding KV caches from previous agents in prefill phases. RelayCaching comprises two core components: (1) a *layer-range profiler* that identifies the critical layer range using the U-shaped profile and selects a detection layer based on inter-layer correlation, and (2) a *token selector* that combines deviation-based and influence-based selection to pinpoint tokens requiring rectification. This design enables RelayCaching to maintain accuracy comparable to full prefilling while rectifying only a small fraction of KV caches, yielding a superior accuracy–efficiency trade-off over existing methods.

In summary, this paper makes the following contributions:

- We systematically characterize KV deviations between decoding and prefill phases, revealing that decoding KV caches exhibit high alignment with full-prefill counterparts despite prefix variation. Residual deviations follow systematic patterns that enable targeted rectification: U-shaped similarity across layers and sparse, inter-layer-correlated deviations across tokens.

- We propose RelayCaching, a training-free method that directly reuses decoding KV caches from upstream agents in downstream prefill phases. RelayCaching employs layer-range profiling to confine rectification to middle layers and token selection to identify critical tokens, achieving efficient cache alignment with minimal overhead.
- We demonstrate that RelayCaching maintains generation quality comparable to full prefilling across diverse tasks while achieving over 80% KV cache reuse and up to $4.7\times$ TTFT reduction in multi-agent systems.

2. Related Works

2.1. LLM-Based Multi-Agent Workflows

LLM-based Multi-Agent Systems have emerged as a prevalent paradigm for solving complex tasks through collective intelligence. Foundational frameworks such as CAMEL (Li et al., 2023) and AutoGen (Wu et al., 2024) pioneered role-playing for task decomposition, while MetaGPT (Hong et al., 2023) introduces Standard Operating Procedures for assembly-line workflows. Recent frameworks like GPTSwarm (Zhuge et al., 2024) and MegaAgent (Wang et al., 2025a) further model agent collectives as dynamic computation graphs. However, these pipelines suffer from *cascading context redundancy*, where upstream outputs are repeatedly reused under changing prefixes, forcing KV cache rebuilding and incurring significant overhead.

2.2. Efficient LLM Inference and KV Caching

KV caching has become the de facto standard for mitigating redundant prefill computation. We categorize existing approaches into prefix caching and pre-computed caching.

Prefix caching reuses KV caches for recurring, identical prefixes. Systems like vLLM (Kwon et al., 2023) and SGLang (Zheng et al., 2024) introduce PagedAttention and RadixAttention to share prefix KV caches across requests. Pre-computed caching relaxes the strict prefix constraint by encoding reusable contexts in advance. Prompt Cache (Gim et al., 2024) enables reuse for frequent prompt modules; Block-Attention (Ma et al., 2025) reuses KV caches for static documents with fine-tuning. KVLink (Yang et al., 2025b) and APE (Yang et al., 2025c) align independently encoded KV caches via trainable tokens or adaptive scaling; CacheBlend (Yao et al., 2025) and EPIC (Hu et al., 2025) correct precomputed caches through dynamic or static token recomputation; DroidSpeak (Liu et al., 2024) extends pre-computed caching across models with shared architectures. Despite these advances, prefix caching requires strict positional alignment that agent-generated contexts rarely satisfy, while pre-computed caching assumes static, offline-encodable content—an assumption violated by dynamically generated outputs, cannot be directly applied in multi-agent settings. These limitations motivate *decode-to-prefill KV reuse*, which transfers KV caches from one agent’s decoding phase to another’s prefilling phase. Unlike pre-computed caching, this paradigm enables targeted rectification of runtime-generated KV caches without prefix alignment constraints, preserving accuracy with minimal overhead.

2.3. Efficiency in Collaborative Inference

Recent efforts to accelerate collaborative inference focus on two orthogonal aspects: topology sparsification and state reuse. Topology sparsification aims to reduce the volume of agent interactions. AgentPrune (Zhang et al., 2025) and AgentDropout (Wang et al., 2025b) optimize message-passing graphs by pruning redundant agents, but treat model inference as a black box. State reuse seeks to minimize computational cost per interaction. To the best of our knowledge, KVCOMM (Ye et al., 2025) is the first to explicitly address *cascading context redundancy* via retrieval-based anchor pools, but relies on external samples and suffers from degraded reuse rates at scale. RelayCaching instead directly exploits the inherent similarity between decoding and prefill KV caches, enabling fine-grained rectification at layer and token granularity without external reference.

3. Observations

In this section, we investigate the alignment and deviation between prefill and decoding KV caches under *decode-to-prefill KV reuse*, where the same tokens appear under different preceding contexts. We address two questions: (1) *Reusability*: are decoding KV caches sufficiently aligned with full-prefill counterparts despite prefix variation? (2) *De-*

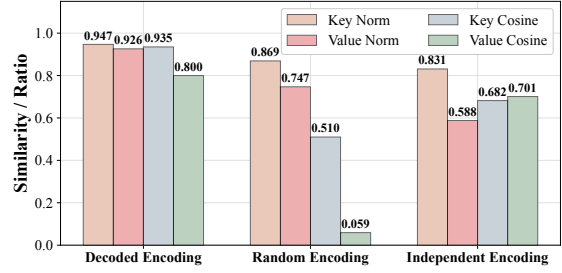


Figure 2. **Macro-level KV similarity.** Average cosine and norm similarity between *Decoding KV* caches and *Full-Prefill KV* caches across layers and tokens on 2WikiMQA, compared against *Random KV* and *Independent KV*.

viation patterns: do residual deviations exhibit systematic patterns that can guide efficient rectification? We conduct experiments on 2WikiMQA (Ho et al., 2020) with Mistral-7B-Instruct-v0.3 (Jiang et al., 2023) using a summarize-then-answer workflow. We further verify that our findings generalize to other model architectures in the Appendix A.

3.1. Macro-Level KV Similarity

We first validate that decoding KV caches remain globally aligned with full-prefill counterparts, finding that *value cosine similarity* emerges as the primary deviation signal. We compare cosine and norm similarity under three settings: (1) *Decoding KV*, produced during decoding with shifted prefixes; (2) *Random KV*, from prefill with the summary replaced by random tokens; and (3) *Independent KV*, from prefill on summary tokens alone without preceding prefix. As shown in Figure 2, *Decoding KV* maintains high macro-level similarity, whereas *Random KV* shows much lower similarity with value cosine approaching zero. Notably, keys in *Decoding KV* align closely with full-prefill keys in both direction and magnitude, whereas values show directional perturbations while largely preserving their magnitudes. In contrast, *Independent KV* exhibits a distinct pattern: value norms deviate substantially from full-prefill counterparts, indicating that pre-computation schemes relying on independently encoded KV caches are incompatible with *decode-to-prefill KV reuse*.

Insights. Decoding KV caches remain highly aligned with full-prefill counterparts, validating the reuse premise. Value cosine similarity is the lowest among all metrics, so we focus on it in subsequent layer- and token-wise analysis.

3.2. Layer-Wise Deviation Pattern

We next analyze how deviations distribute across layers, finding that middle layers exhibit the largest deviations and contribute most to subsequent generation errors. Figure 3a reveals a characteristic *U-shaped profile* in value cosine similarity: high in shallow layers, dropping to a minimum in middle layers, and partially recovering in deeper lay-

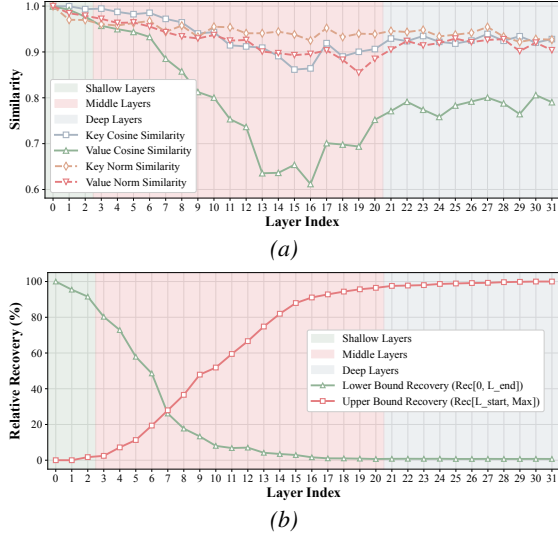


Figure 3. **Layer-wise structured deviations.** (a) Layer-wise similarity between decoding and full-prefill KV caches, highlighting a U-shaped value cosine similarity profile across layers. (b) Relative recovery of value cosine similarity for tokens after the reuse position when decoding KV caches are replaced with their full-prefill counterparts either up to or from a given layer index.

ers. To verify that middle-layer deviations are the primary error source for subsequent generation, we perform an oracle experiment that selectively substitutes decoding KV caches with full-prefill KV at different layer ranges. Figure 3b reports the resulting value cosine similarity for tokens following the reused segment under two settings: (i) substituting all layers up to index i , and (ii) substituting all layers from i onward. In both cases, similarity recovery is steepest when the substituted range covers the middle layers identified by the U-shaped profile, with diminishing returns when extending into shallow or deep layers, confirming that middle layers dominate downstream generation quality.

Insights. The U-shaped profile provides a principled criterion for identifying the critical layer range, enabling targeted rectification that captures most of the benefit while bypassing shallow and deep layers.

3.3. Token-Wise Deviation Pattern

Finally, we investigate how deviations distribute across tokens, finding that token-level deviations are sparse and exhibit strong inter-layer correlation. As shown in Figure 4a, only a small subset of positions exhibits large deviations, while most tokens maintain high similarity with full-prefill counterparts. Some of these high-deviation tokens are position-dependent, such as tokens near the start of the reuse segment, which are inherently more sensitive to prefix variation; others are content-dependent, reflecting input-specific sensitivity to the preceding context. Figure 4b shows that the Spearman rank correlation of token-wise deviations between adjacent layers starts low but rapidly

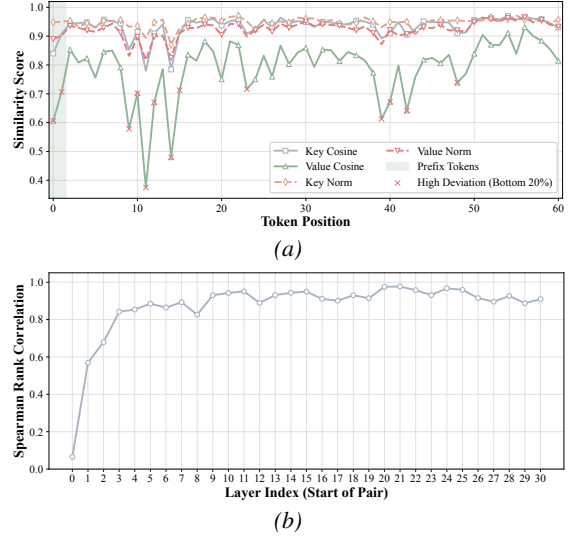


Figure 4. **Token-wise structured deviations.** (a) Token-wise similarity between decoding and full-prefill KV caches, averaged over layers, revealing a sparse set of high-deviation positions. (b) Spearman rank correlation of token-wise value cosine deviations between adjacent layers, showing that high-deviation positions, once emerged, tend to persist.

increases and plateaus, indicating that high-deviation positions persist once they emerge.

Insights. The combination of token-wise sparsity and inter-layer correlation implies that rectification can be highly selective: by identifying high-deviation tokens at a layer where rankings have stabilized, we can propagate this selection to subsequent layers, achieving most of the rectification benefit at minimal cost.

4. RelayCaching

4.1. Overall Architecture

Guided by Section 3, we present RelayCaching, a training-free method that accelerates multi-agent collaboration by reusing decoding KV caches with targeted rectification. The key insight is that deviations concentrate in middle layers and a sparse set of tokens, enabling accurate repair at these position instead of full-prefill recomputation. As depicted in Figure 5, RelayCaching comprises two components: a **layer-range profiler** that exploits the U-shaped similarity profile and inter-layer correlation identifies the critical layer range $[L_{start}, L_{end}]$ and a detection layer L_{det} ; and a **token selector** that combines deviation-based and influence-based criteria. Together, these enable selective recomputation of identified tokens within the critical range, preserving generation quality at a fraction of the cost.

4.2. Layer-Range Profiler

The layer-range profiler determines *where* to apply rectification and *at which layer* to perform token selection. The

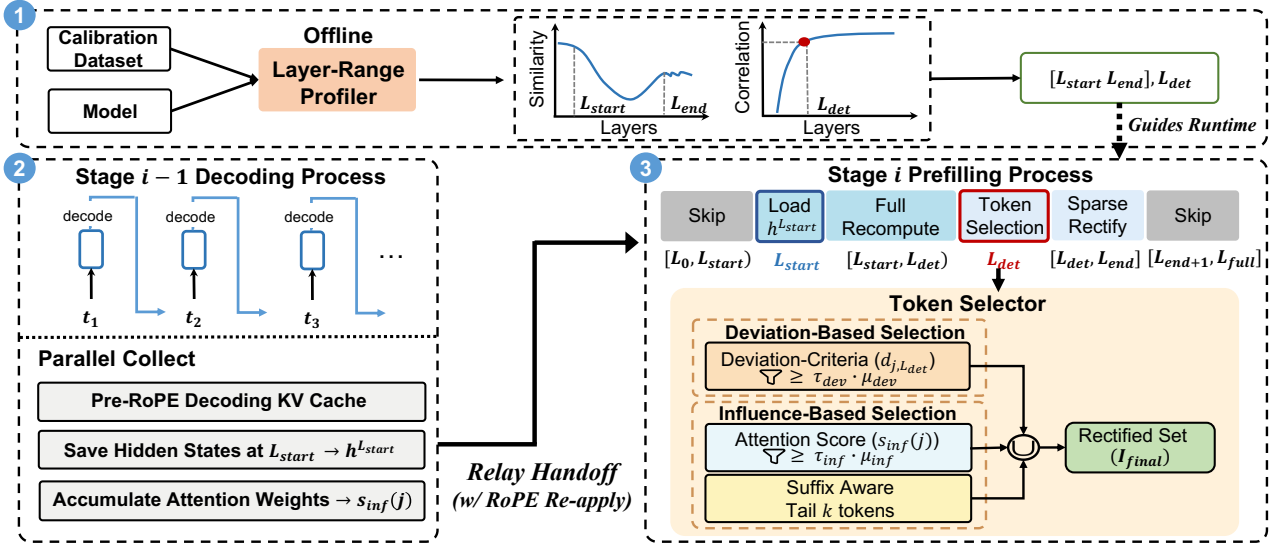


Figure 5. **Overview of RelayCaching.** (1) An offline layer-range profiler analyzes calibration data to identify a critical layer range $[L_{\text{start}}, L_{\text{end}}]$ from the similarity profile and a detection layer L_{det} from the inter-layer correlation profile. (2) During stage- $(i-1)$ decoding, the cache manager stores pre-RoPE KV caches, saves hidden states at L_{start} , and accumulates influence scores $s_{\text{inf}}(j)$. (3) At stage- i prefilling, RelayCaching re-applies RoPE to align reused KV caches to new positions, performs token selection at L_{det} via combined deviation- and influence-based criteria, and finally applies sparse rectification on selected tokens within $[L_{\text{start}}, L_{\text{end}}]$.

U-shaped profile (Section 3.2) suggests rectification should concentrate on middle layers $[L_{\text{start}}, L_{\text{end}}]$, while inter-layer correlation (Section 3.3) enables identifying a detection layer L_{det} for reliable token selection. The profiler performs offline profiling on auxiliary datasets to determine these parameters. Let $\mathbf{I}_{\text{reuse}}$ denote the set of token positions in the reuse segment. To quantify the alignment between reused and full-prefill KV caches, we define the token-wise deviation at position j and layer ℓ as

$$d_{j,\ell} = 1 - \frac{1}{H} \sum_{h=1}^H \cos(\mathbf{v}_{h,j,\ell}^{\text{reuse}}, \mathbf{v}_{h,j,\ell}^{\text{full}}), \quad (1)$$

where H is the number of heads, and $\mathbf{v}_{h,j,\ell}^{\text{reuse}}, \mathbf{v}_{h,j,\ell}^{\text{full}}$ denote the value vectors of head h at position j in layer ℓ under reused decoding and full-prefill, respectively. The layer-wise similarity is then the average over all reused tokens:

$$s_\ell = \frac{1}{|\mathbf{I}_{\text{reuse}}|} \sum_{j \in \mathbf{I}_{\text{reuse}}} (1 - d_{j,\ell}). \quad (2)$$

Start layer. Since shallow layers already exhibit high similarity and require no rectification, they can provide a high-fidelity starting point for subsequent computation. We set L_{start} to the deepest layer with $s_\ell \geq \tau_{\text{st}}$. At inference time, we load cached hidden states $\mathbf{h}^{(L_{\text{start}})}$ as the starting point for rectification rather than recomputing shallow layers.

End layer. Deeper layers enter a stable regime where deviations partially recover and additional rectification yields diminishing returns on output quality. We locate the onset

of this regime by scanning forward from the minimum-similarity layer $\ell_{\min} = \arg \min_\ell s_\ell$. We estimate a baseline $b = \mu_{\text{last}} - \sigma_{\text{last}}$, where μ_{last} and σ_{last} are the mean and standard deviation of s_ℓ over the last T layers. L_{end} is set to the first layer followed by C consecutive layers satisfying

$$s_\ell \geq b \text{ and } |s_\ell - s_{\ell-1}| < \lambda \sigma_{\text{last}}. \quad (3)$$

Detection layer. While the layer range defines *where* to rectify, we also need to determine *at which layer* to select tokens to rectify. Detecting tokens at L_{start} may lack discriminative power due to high similarity in shallow layers. To address this, we identify a detection layer, L_{det} within $[L_{\text{start}}, L_{\text{end}}]$ where deviation patterns have stabilized. We examine the Spearman rank correlation ρ_ℓ of token-wise deviations between adjacent layers. We define the second-order difference of the correlation trend as:

$$\alpha_\ell = \rho_\ell - 2\rho_{\ell-1} + \rho_{\ell-2} \quad (4)$$

We then identify ℓ^* as the first layer where α_ℓ transitions from positive to negative, indicating that correlation growth has begun to decelerate. The detection layer is set to $L_{\text{det}} = \ell^* + 1$, where the deviation ranking has fully stabilized.

At inference time, layers outside $[L_{\text{start}}, L_{\text{end}}]$ directly reuse decoding KV caches without rectification. Within this range, layers in $[L_{\text{start}}, L_{\text{det}}]$ are fully recomputed starting from cached hidden states at L_{start} to enable deviation detection, while layers in $[L_{\text{det}}, L_{\text{end}}]$ apply sparse token-selective rectification only on selected tokens.

4.3. Token Selection Module

Section 3.3 reveals that deviations are *sparse* and *inter-layer correlated*, enabling single-pass token identification at L_{det} that propagates throughout $[L_{\text{det}}, L_{\text{end}}]$. Beyond deviation magnitude, a token’s importance also depends on its *downstream influence*: moderately deviated tokens can substantially impact outputs if subsequent tokens repeatedly attend to them. Accordingly, RelayCaching selects tokens using two criteria: Deviation-based selection targets tokens with large $d_{j, L_{\text{det}}}$; Influence-based selection prioritizes tokens receiving high attention from subsequent positions. Their union forms the final rectification set.

4.3.1. DEVIATION-BASED SELECTION

Deviation-based selection targets tokens with large directional mismatches at L_{det} . Using $s_{\text{dev}}(j) = d_{j, L_{\text{det}}}$ and mean μ_{dev} , we select:

$$\mathbf{I}_{\text{dev}} = \{j \in \mathbf{I}_{\text{reuse}} \mid s_{\text{dev}}(j) \geq \tau_{\text{dev}} \cdot \mu_{\text{dev}}\}, \quad (5)$$

where τ_{dev} is a scaling factor. This mean-relative threshold adapts to deviation magnitude. Due to inter-layer correlation, this selection is reused throughout $[L_{\text{det}}, L_{\text{end}}]$.

4.3.2. INFLUENCE-BASED SELECTION

Deviation-based selection may miss tokens with moderate deviation but substantial downstream influence. We incorporate influence-based selection to identify such tokens.

Attention-derived importance. We measure influence by accumulating attention weights received across all layers and decoding steps:

$$s_{\text{inf}}(j) = \sum_{t, \ell, h} \alpha_{t, \ell, h, j}, \quad (6)$$

where $\alpha_{t, \ell, h, j}$ is the attention weight at decoding step t , layer ℓ , head h . We select influential tokens via

$$\mathbf{I}_{\text{inf-score}} = \{j \in \mathbf{I}_{\text{reuse}} \mid s_{\text{inf}}(j) \geq \tau_{\text{inf}} \cdot \mu_{\text{inf}}\}, \quad (7)$$

where μ_{inf} is the mean influence and τ_{inf} is a scaling factor.

Suffix-aware correction. Attention-derived importance is inherently biased toward early positions. Meanwhile, suffix tokens near the reuse segment boundary often exert disproportionate influence on newly appended content due to attention locality. To balance this, we additionally include the last K_{suf} trailing tokens as $\mathbf{I}_{\text{inf-suffix}}$, yielding:

$$\mathbf{I}_{\text{inf}} = \mathbf{I}_{\text{inf-score}} \cup \mathbf{I}_{\text{inf-suffix}}. \quad (8)$$

The final set combines both criteria:

$$\mathbf{I}_{\text{final}} = \mathbf{I}_{\text{dev}} \cup \mathbf{I}_{\text{inf}}. \quad (9)$$

This dual-criterion selection maintains a small rectification set while effectively correcting deviations, achieving a favorable accuracy–efficiency trade-off.

5. Evaluation

This section presents analyses to answer the following research questions: **RQ1**: Can RelayCaching maintain generation quality comparable to full prefilling? **RQ2**: Can RelayCaching improve efficiency for real-world multi-agent tasks? **RQ3**: How does each component contribute to the efficiency-accuracy trade-off? **RQ4**: How sensitive is RelayCaching to key hyperparameters?

5.1. Experiment Setup

Datasets and Workflows. For accuracy evaluation, following KVCOMM (Ye et al., 2025), we construct fully connected multi-agent systems on three benchmarks: GSM8K (Cobbe et al., 2021) (math reasoning), HumanEval (Chen, 2021) (coding), and MMLU (Hendrycks et al., 2020) (knowledge QA). For efficiency profiling, we use AIME24 (MAA, 2025) to produce long reasoning traces that stress-test long-context efficiency. Detailed agent configurations are in Appendix B.4.

Models and Implementation. We employ Llama-3.1-8B-Instruct (Dubey et al., 2024) for reasoning and knowledge tasks, Qwen2.5-Coder-7B-Instruct (Hui et al., 2024) for coding, and Qwen3-0.6B (Yang et al., 2025a) for efficiency experiments. The layer-range profiler is calibrated on 2WikiMQA (Ho et al., 2020) to ensure task-agnostic deviation detection. Default hyperparameters: $\tau_{\text{down}}=1.45$, $\tau_{\text{diff}}=1.5$, $L_{\text{suf}}=10$. All models run in BF16 on a single NVIDIA H20 GPU.

Baselines. We compare against two reference baselines: *Full-prefill* (FULL) without reuse, and *Direct Reuse* (ZERO) without rectification. We further include three state-of-the-art methods: CacheBlend (Yao et al., 2025) and EPIC (Hu et al., 2024) for pre-computed KV caching, and KVCOMM (Ye et al., 2025) for multi-agent reuse. All methods use recommended settings; see Appendix B.3 for details.

Metrics. We report task-specific accuracy (pass@1 for HumanEval, accuracy for MMLU and GSM8K) and *Reuse Rate*, the fraction of KV entries reused.

5.2. Main Results

Table 1 and Figure 6 present comparative results across GSM8K, MMLU, and HumanEval. RelayCaching consistently matches the Full-Prefill baseline while maintaining reuse rates of over 80% across most multi-agent settings. Notably, the layer-range detected on 2WikiMQA generalizes across all tasks without per-task tuning. Pre-computed caching methods such as EPIC and CacheBlend struggle with the dynamic prefix variations in decode-to-prefill reuse: EPIC’s static prefix recomputation fails to adapt to content-sensitive deviations, with accuracy dropping to 58.73% on

RelayCaching

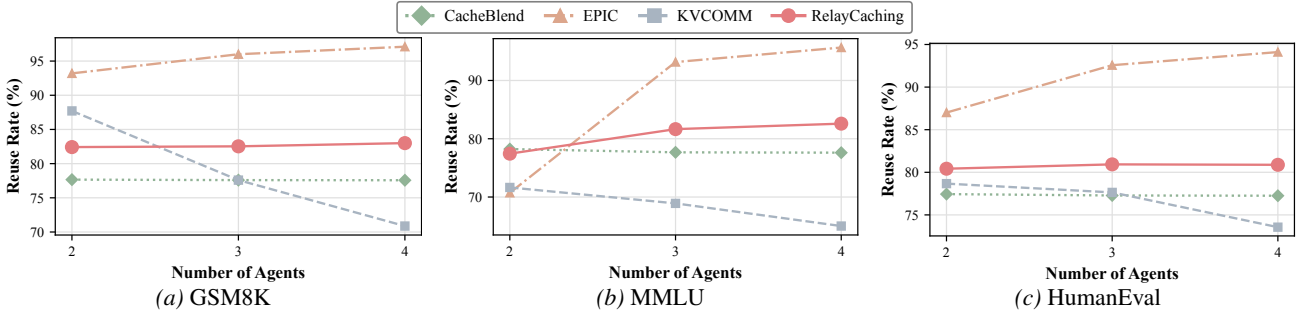


Figure 6. **Computational reuse efficiency across diverse benchmarks.** We compare the Reuse Rate of RelayCaching against baselines on (a) GSM8K, (b) MMLU, and (c) HumanEval as the number of agents increases.

Table 1. **RelayCaching consistently maintains performance comparable to the Full-prefill.** Columns indicate the number of agents in the workflow. **Bold** indicates the best performance; underline marks the second best.

Dataset	Method	2 Agents	3 Agents	4 Agents
<i>Llama-3.1-8B-Instruct (Accuracy %)</i>				
GSM8K	Full Prefill	85.97	84.69	84.69
	Zero Reuse	40.33	40.03	53.68
	CacheBlend	76.72	77.56	74.73
	EPIC	58.73	66.46	58.42
	KVComm	<u>84.23</u>	<u>82.34</u>	<u>84.91</u>
	RelayCaching	84.84	85.50	85.17
MMLU	Full Prefill	71.24	73.20	66.67
	Zero Reuse	67.97	41.18	35.29
	CacheBlend	69.28	64.71	<u>66.67</u>
	EPIC	69.28	45.10	43.79
	KVComm	<u>68.63</u>	<u>69.28</u>	66.01
	RelayCaching	69.28	71.90	67.97
<i>Qwen2.5-Coder-7B-Instruct (Pass@1 %)</i>				
HumanEval	Full Prefill	82.61	85.71	87.58
	Zero Reuse	81.99	85.09	86.96
	CacheBlend	<u>82.61</u>	<u>84.47</u>	85.71
	EPIC	81.99	83.23	<u>86.34</u>
	KVComm	<u>82.61</u>	83.23	85.09
	RelayCaching	82.61	85.09	87.58

GSM8K; CacheBlend’s value-norm-based detection does not capture the cosine similarity that governs decode-to-prefill KV deviation patterns, resulting in accuracy below 78% on GSM8K and MMLU. KVCOMM achieves competitive accuracy but relies on similarity-based retrieval from anchor pools, causing reuse rates to decline from 88% to 70% in GSM8K as context diversity grows. RelayCaching maintains stable reuse from two to four agents while matching or exceeding KVCOMM’s accuracy. In summary, RelayCaching bridges the accuracy–efficiency gap, achieving Full-Prefill accuracy with over 80% reuse rate.

5.3. Efficiency Evaluation

We evaluate the system efficiency of RelayCaching on the AIME dataset using Qwen3-0.6B. Our analysis focuses on

Table 2. **Per-agent TTFT breakdown and speedup.** Prefix: 512 tokens; Output: 2,048 tokens; Model: Qwen3-0.6B.

Method	Agent 2	Agent 3	Agent 4	Agent 5
Full Prefill	85.2	180.2	321.5	493.9
CacheBlend	44.9	76.7	112.5	159.3
Speedup	1.90×	2.35×	2.86×	3.10×
RelayCaching	40.6	57.0	73.2	104.8
Full recompute	1.6	2.8	4.3	7.9
Index selection	0.5	0.6	0.7	0.8
Speedup	2.10×	3.16×	4.39×	4.71×

TTFT acceleration and its scalability across varying numbers of agents and reused sequence lengths.

Latency Breakdown Analysis. Table 2 decomposes the TTFT latency. As context accumulates from Agent 2 to Agent 5, full prefill latency increases from 85.2 ms to 493.9 ms (5.8×), reflecting the quadratic complexity of prefill computation. In contrast, RelayCaching’s latency grows from 40.6 ms to only 104.8 ms (2.6×), achieving a 4.71× speedup at Agent 5. The breakdown reveals favorable scaling characteristics. Index selection grows minimally from 0.5 to 0.8 ms across agents, validating the efficiency of our token selector. The full-recompute cost (1.6–7.9 ms) represents the primary overhead our method yet remains negligible compared to full prefill. Overall, RelayCaching’s latency scales at 2.6×, compared to 5.8× for Full Prefill and 3.5× for CacheBlend, demonstrating sub-linear scaling as the number of agents increases.

Scalability with Context Length. To assess performance in long-context regimes, we sweep the cumulative output length from 512 to 12,288 tokens among three collaborating agents. Figure 8 reports the average per-agent TTFT. Standard full prefill exhibits quadratic complexity, with latency escalating to 3.89 s at 12k tokens. While CacheBlend mitigates this growth, it still reaches 1.08 s because it re-computes a fixed proportion of tokens, causing its cost to scale linearly with context length. In contrast, RelayCaching exhibits sub-linear scaling, recording only 0.42 s at the maximum length—a 9.2× speedup over full prefill and 2.5× over CacheBlend. This sub-linear scalability stems from the sparsity of token-wise deviations (Section 3.3): as con-

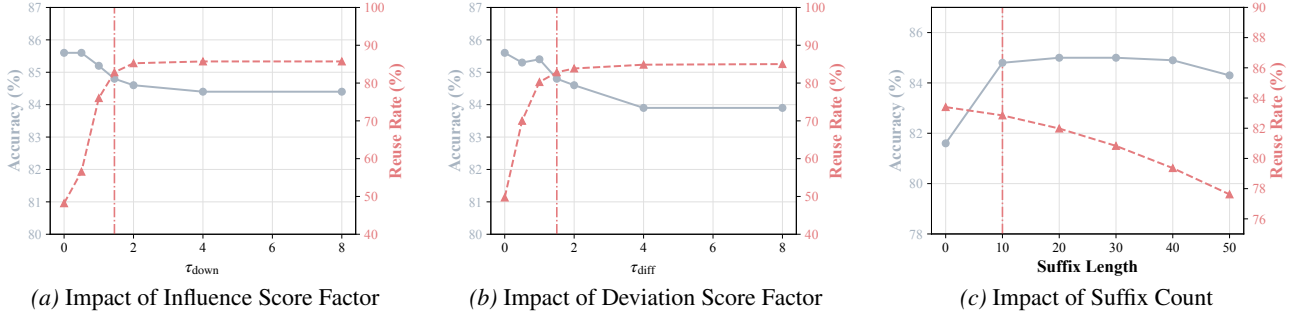


Figure 7. **Ablation on key RelayCaching hyperparameters.** We analyze the trade-off between Accuracy (left axis) and Reuse Rate (right axis) by varying: (a) the influence score factor τ_{down} , (b) the deviation score factor τ_{diff} , and (c) the suffix length L_{suf} . Red markers denote the default configuration used in the main experiments.

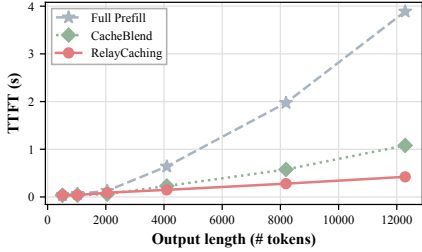


Figure 8. **Scalability of TTFT with context growth.** We measure the average TTFT of downstream agents as the cumulative output length scales from 512 to 12,288 tokens.

Table 3. Ablation study on GSM8K (Llama-3.1-8B-Instruct).

Method	Reuse (%)	Acc (%)
Full Prefill	0.00	85.97
+ Layer Range Profiler	46.88	85.90
+ $\mathbf{I}_{\text{dev}}$	90.37	79.68
+ $\mathbf{I}_{\text{inf-score}}$	85.84	81.65
+ $\mathbf{I}_{\text{inf-suffix}}$ (Ours)	85.35	84.84

text grows, the number of high-deviation tokens increases more slowly than the total sequence length, keeping the rectification cost sub-linear.

5.4. Ablation Results

We conduct a fine-grained ablation on GSM8K under 2 agents setting (Table 3). Restricting rectification to the critical layer range yields 85.90% accuracy but a modest 46.88% reuse rate. Adding deviation-based selection $\mathbf{I}_{\text{dev}}$ boosts reuse to 90.37% but degrades accuracy to 79.68%, indicating that deviation metrics alone overlook generation-critical tokens. Further incorporating score-based influence selection $\mathbf{I}_{\text{inf-score}}$ improves accuracy to 81.65% at the cost of lower reuse (85.84%), confirming that influence-aware criteria capture tokens missed by deviation alone. The full RelayCaching, integrating all three strategies, achieves 84.84% accuracy while maintaining 85.35% reuse. This shows that multi-dimensional selection is essential for preserving generation quality.

5.5. Sensitive Analysis

We study the impact of the main RelayCaching hyperparameters on the accuracy–reuse trade-off on GSM8K with 2 agents, as shown in Figure 7. Each panel varies one hyperparameter while keeping the others fixed at their default values. Panels (a) and (b) exhibit consistent patterns: increasing either τ_{down} or τ_{diff} causes the reuse rate to rise sharply before plateauing, while accuracy varies by 2%. This indicates that the majority of tokens contribute minimally to deviation correction. Since the steepest trade-off occurs between values of 1 and 2, we select $\tau_{\text{down}} = 1.45$ and $\tau_{\text{diff}} = 1.5$ to maximize reuse with minimal accuracy degradation. Panel (c) varies the suffix length L_{suf} used for recovery. A modest suffix length (e.g., 10) suffices to recover most of the deviation in suffix position, while longer suffixes yield diminishing returns at the cost of reduced reuse.

6. Conclusion

We introduced RelayCaching, a training-free framework that accelerates multi-agent LLM collaboration by reusing decoding KV caches for subsequent prefilling. Our systematic analysis revealed that decoding KV caches remain highly aligned with their full-prefill counterparts despite prefix variation, with value cosine similarity serving as the primary deviation indicator. Residual deviations exhibit systematic patterns: a U-shaped layer-wise similarity profile where middle layers show the largest deviations and dominate subsequent generation quality, and token-wise sparsity with strong inter-layer correlation. Leveraging these insights, RelayCaching employs a layer-range profiler to confine rectification to a critical layer range, and a token selector combining deviation-based and influence-based criteria to identify a sparse set of tokens requiring rectification. Experiments across reasoning, coding, and knowledge benchmarks demonstrate RelayCaching achieves over 80% KV cache reuse and up to $4.7\times$ TTFT speedup while preserving accuracy comparable to full prefilling, confirming the effectiveness of decoding-to-prefill KV reuse for efficient multi-agent collaboration.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Chen, M. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., Fan, A., et al. The llama 3 herd of models. *arXiv e-prints*, pp. arXiv–2407, 2024.
- Gim, I., Chen, G., Lee, S.-s., Sarda, N., Khandelwal, A., and Zhong, L. Prompt cache: Modular attention reuse for low-latency inference. *Proceedings of Machine Learning and Systems*, 6:325–338, 2024.
- Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.
- Ho, X., Nguyen, A.-K. D., Sugawara, S., and Aizawa, A. Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps. *arXiv preprint arXiv:2011.01060*, 2020.
- Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Wang, J., Zhang, C., Wang, Z., Yau, S. K. S., Lin, Z., et al. Metagpt: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations*, 2023.
- Hu, J., Huang, W., Wang, H., Wang, W., Hu, T., Zhang, Q., Feng, H., Chen, X., Shan, Y., and Xie, T. Epic: Efficient position-independent context caching for serving large language models. *arXiv e-prints*, pp. arXiv–2410, 2024.
- Hu, J., Huang, W., Wang, W., Wang, H., Feng, H., Chen, X., Shan, Y., Xie, T., et al. Epic: Efficient position-independent caching for serving large language models. In *Forty-second International Conference on Machine Learning*, 2025.
- Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., Liu, T., Zhang, J., Yu, B., Lu, K., et al. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.
- Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., de las Casas, D., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., Lavaud, L. R., Lachaux, M.-A., Stock, P., Scao, T. L., Lavril, T., Wang, T., Lacroix, T., and Sayed, W. E. Mistral 7b, 2023. URL <https://arxiv.org/abs/2310.06825>.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pp. 611–626, 2023.
- Li, G., Hammoud, H., Itani, H., Khizbullin, D., and Ghanem, B. Camel: Communicative agents for” mind” exploration of large language model society. *Advances in Neural Information Processing Systems*, 36:51991–52008, 2023.
- Liu, Y., Huang, Y., Yao, J., Feng, S., Gu, Z., Du, K., Li, H., Cheng, Y., Jiang, J., Lu, S., et al. Droidspeak: Kv cache sharing for cross-llm communication and multi-llm serving. *arXiv preprint arXiv:2411.02820*, 2024.
- Ma, D., Wang, Y., and Lan, T. Block-attention for efficient prefilling. In *The Thirteenth International Conference on Learning Representations*, 2025.
- MAA. American invitational mathematics examination (aime). <https://maa.org/math-competitions/aime>, 2025. Mathematics Competition Series.
- Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., Su, Y., Cong, X., et al. Chatdev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 15174–15186, 2024.
- Su, H., Chen, R., Tang, S., Yin, Z., Zheng, X., Li, J., Qi, B., Wu, Q., Li, H., Ouyang, W., et al. Many heads are better than one: Improved scientific idea generation by a llm-based multi-agent system. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 28201–28240, 2025.
- Wang, Q., Wang, T., Tang, Z., Li, Q., Chen, N., Liang, J., and He, B. Megaagent: A large-scale autonomous llm-based multi-agent system without predefined sops. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 4998–5036, 2025a.
- Wang, Z., Wang, Y., Liu, X., Ding, L., Zhang, M., Liu, J., and Zhang, M. AgentDropout: Dynamic agent elimination for token-efficient and high-performance LLM-based multi-agent collaboration. In Che, W., Nabende,

- J., Shutova, E., and Pilehvar, M. T. (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 24013–24035, Vienna, Austria, July 2025b. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.1170. URL <https://aclanthology.org/2025.acl-long.1170/>.
- Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First Conference on Language Modeling*, 2024.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025a.
- Yang, J., Hou, B., Wei, W., Bao, Y., and Chang, S. Kvlink: Accelerating large language models via efficient kv cache reuse. *arXiv preprint arXiv:2502.16002*, 2025b.
- Yang, X., Chen, T., and Chen, B. Ape: Faster and longer context-augmented generation via adaptive parallel encoding. In *The Thirteenth International Conference on Learning Representations*, 2025c.
- Yao, J., Li, H., Liu, Y., Ray, S., Cheng, Y., Zhang, Q., Du, K., Lu, S., and Jiang, J. Cacheblend: Fast large language model serving for rag with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*, pp. 94–109, 2025.
- Ye, H., Gao, Z., Ma, M., Wang, Q., Fu, Y., Chung, M.-Y., Lin, Y., Liu, Z., Zhang, J., Zhuo, D., et al. Kvcomm: Online cross-context kv-cache communication for efficient llm-based multi-agent systems. *arXiv preprint arXiv:2510.12872*, 2025.
- Zhang, G., Yue, Y., Li, Z., Yun, S., Wan, G., Wang, K., Cheng, D., Yu, J. X., and Chen, T. Cut the crap: An economical communication pipeline for llm-based multi-agent systems. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Zheng, L., Yin, L., Xie, Z., Sun, C. L., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., et al. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37:62557–62583, 2024.
- Zhuge, M., Wang, W., Kirsch, L., Faccio, F., Khizbullin, D., and Schmidhuber, J. Gptswarm: Language agents as optimizable graphs. In *Forty-first International Conference on Machine Learning*, 2024.

A. Further Observations

In Section 3, we presented the systematic deviation patterns of decoding-to-prefill KV reuse using Mistral-7B-Instruct-v0.3. To verify the universality of these phenomena across different model architectures, sizes, and capabilities, we extend our analysis to five additional state-of-the-art models: Llama-3.1-8B-Instruct, Qwen2.5-Coder-7B-Instruct, DeepSeek-R1-Distill-Qwen-32B, Qwen3-30B-A3B, and the lightweight Qwen3-0.6B.

Universality of Layer-Wise Patterns. Figure 9 illustrates the layer-wise cosine similarity profiles and relative recovery curves for these models. Consistent with our main observations, all models exhibit a characteristic U-shaped value cosine similarity profile. The "middle layers"—where reasoning and context integration primarily occur—consistently show the highest sensitivity to prefix shifts (lowest similarity), while shallow layers maintain relatively high fidelity and deep layers only partially recover.

Furthermore, the recovery curves confirm that this deviation is structured: rectifying the specific range of middle layers identified by the U-curve yields the steepest similarity recovery across all models. For instance, even in the larger DeepSeek-R1-Distill-Qwen-32B or the architectural variant Qwen3-30B-A3B, the strategy of prioritizing middle-layer rectification remains the most effective.

Universality of Token-Wise Patterns. Figure 10 depicts the token-wise deviation distribution and inter-layer rank correlations. Across all evaluated models, we observe structured sparsity: only a small subset of tokens exhibits significant value deviation, while the majority remain highly aligned with full-prefill states. Crucially, the rank correlation analysis reveals that deviations are not random noise. The high Spearman rank correlation between adjacent layers (rapidly stabilizing > 0.9) indicates that once a token is corrupted by prefix shifts in early layers, it tends to remain a high-deviation outlier in subsequent layers. This persistence is observed universally, from the 0.6B model to the 32B model, validating the robustness of our selection-based rectification strategy.

B. Experiment Details

In this section, we provide detailed configurations for our observation experiments, baseline implementations, and the main evaluation setup to ensure reproducibility.

B.1. Setup for Observation Experiment

To investigate the alignment between decoding and prefilling KV caches (Section 3), we constructed a controlled *Summarize-then-Answer* pipeline using the 2WikiMQA dataset (Ho et al., 2020).

Data Construction. We utilized the validation set processed into a JSON format. The dataset consists of 200 samples. For each instance, we formulated a two-stage process:

1. **Stage 1 (Decoding):** The model generates a summary for a given Wikipedia passage. The maximum number of generated tokens is set to 512. The KV caches generated during this decoding phase are stored as the *Decoding KV*.
2. **Stage 2 (Prefilling):** The generated summary is concatenated with a new prefix (derived from the question and task). We perform a full prefill computation on this sequence to obtain the ground-truth *Full-Prefill KV* cache.

Configuration. We utilized Mistral-7B-Instruct-v0.3 as the primary probe model for the observation analysis presented in the main text (Section 3). Additional models (e.g., Qwen, Llama, DeepSeek) analyzed in Appendix A follow the same pipeline. All experiments were conducted in bfloat16 precision with greedy decoding (temperature = 0).

B.2. RelayCaching Configuration

Layer Selection. The recompute layer range $[L_{\text{start}}, L_{\text{end}}]$ is determined offline using the profiling algorithm described in Section 4.1. The hyperparameters for this selection process are robust across models. We list the default values used in our experiments in Table 4.

Token Selection. For the runtime token rectification, we adopt the following default settings based on our sensitivity studies (Figure 7). These values were selected to maximize the reuse rate while maintaining generation quality across both reasoning and coding tasks:

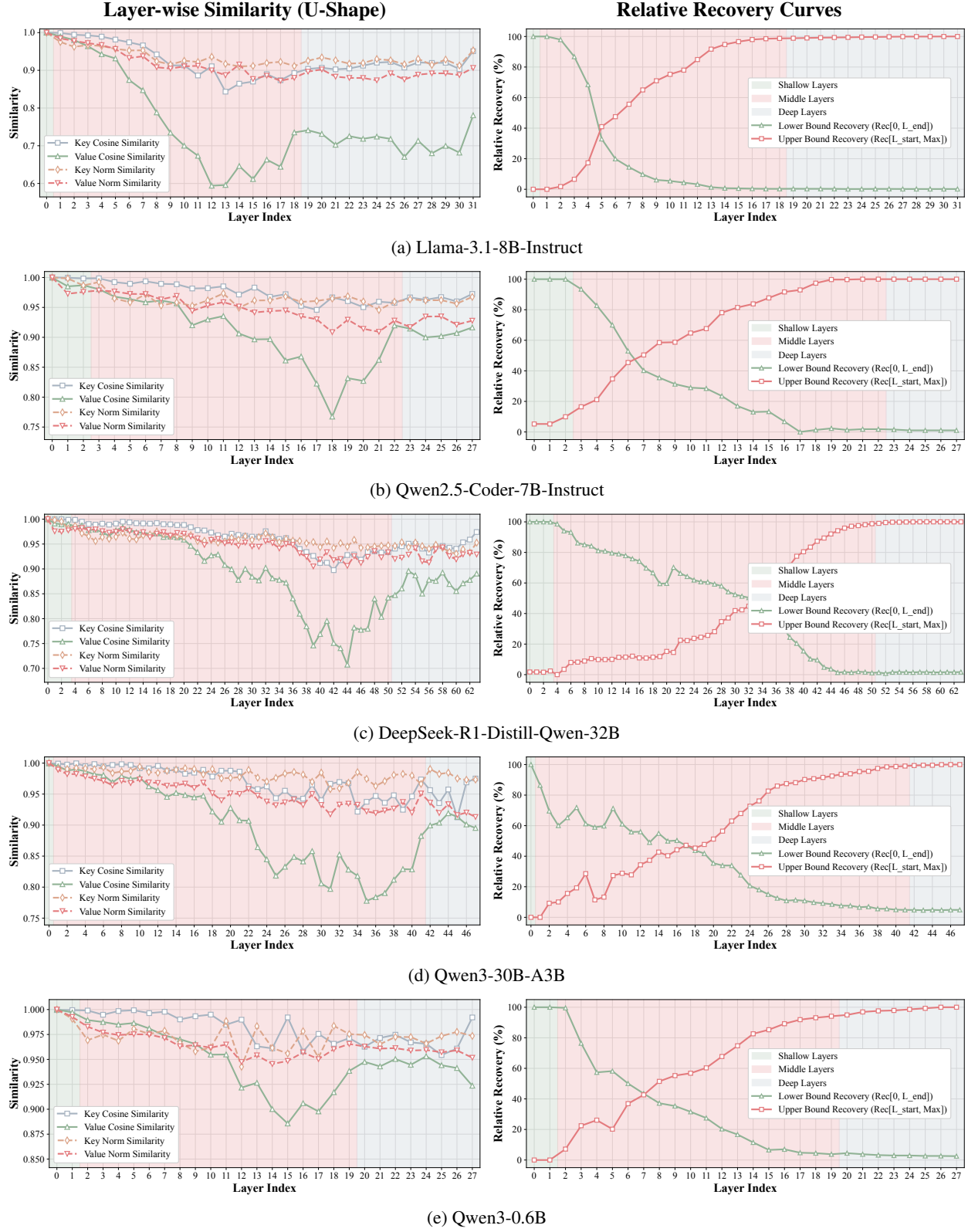


Figure 9. Universality of layer-wise structured deviations. **Left Column:** Layer-wise cosine similarity and norm similarity profiles. The distinct U-shaped drop in value cosine similarity (red lines) is consistent across diverse models. **Right Column:** Relative recovery curves. In all cases, recomputing the middle layers (where the similarity drops) yields the most efficient recovery of downstream token fidelity.

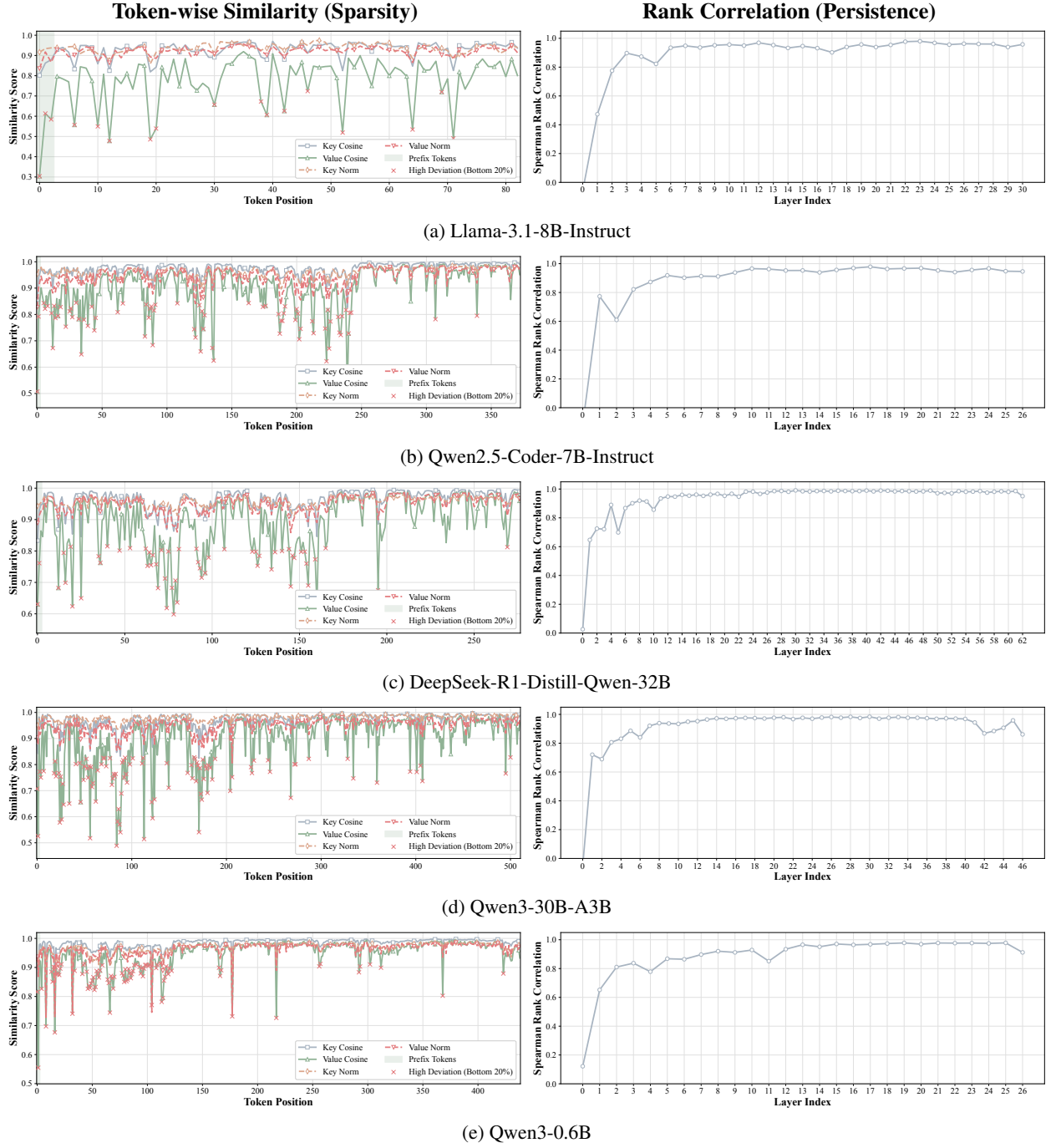


Figure 10. Universality of token-wise structured deviations. **Left Column:** Token-wise similarity profiles averaged over middle layers. The deviations are consistently sparse, with most tokens retaining high fidelity. **Right Column:** Spearman rank correlation of value cosine deviation between adjacent layers. The high correlation confirms that deviation patterns are persistent and predictable across depths.

Table 4. Hyperparameters for Critical Layer Selection. These values are used to identify the deviation-prone middle layers based on the U-shaped similarity profile.

Symbol	Description	Default
τ_{low}	Lower similarity threshold	0.99
T	Right-tail layers	5
λ	Stable multiplier	2.0
N_{min}	Min rise samples	3
C	Consecutive stables	2

- **Difference Threshold (τ_{diff}):** Set to 1.5. Tokens with a normalized KV difference score above this threshold times the mean deviation score are identified as high-deviation outliers and selected for rectification.
- **Downstream Threshold (τ_{down}):** Set to 1.45. This threshold filters tokens based on their cumulative attention influence, ensuring that semantically pivotal tokens are rectified.
- **Suffix Length (L_{suf}):** Set to 10. We empirically found that rectifying the last 10 tokens of the relay segment is sufficient to correct local attention patterns and mitigate boundary artifacts.

Model-Specific Layer Configurations. The specific critical layer boundaries identified by our profiling algorithm vary across model architectures. Table 5 details the configurations used in our experiments. Specifically, L_{start} denotes the reuse hidden layer where we begin injecting decoded states, and L_{det} marks the detection layer used for computing difference metrics. L_{end} marks the end of the critical range.

Table 5. Model-specific layer configurations derived from offline profiling.

Model	L_{start}	L_{det}	L_{end}
Llama-3.1-8B-Instruct	1	3	18
Qwen2.5-Coder-7B-Instruct	3	4	22
Qwen3-0.6B	2	3	19

B.3. Baseline Descriptions and Implementation

We compare RelayCaching against performance bounds and state-of-the-art KV reuse methods. All baselines are implemented using the Hugging Face Transformers library.

Performance Bounds:

- **Full-prefill (FULL):** The standard attention mechanism that performs complete prefilling for all input tokens, establishing the upper bound for generation quality.
- **Direct Reuse (ZERO):** A naive baseline that directly grafts previously computed KV states without any rectification, defining the theoretical upper bound for efficiency. This typically suffers from accuracy degradation due to unaddressed deviation patterns.

KV Reuse Methods:

- **CacheBlend (Yao et al., 2025):** A selective recomputation method that identifies high-deviation tokens based on value norm deviation in second layer. We set the recomputation token size K dynamically to the top $\alpha = 20\%$ of the relay segment length ($K = \lfloor N_{\text{relay}} \times 0.2 \rfloor$). In long-context scenarios, this linear scaling overhead ($K \propto N$) significantly limits the end-to-end speedup compared to RelayCaching’s sparse selection strategy.
- **EPIC (Hu et al., 2024):** A position-independent caching method that recomputes only a fixed set of prefix tokens. We implement the LegoLink algorithm with a chunk size of 16. However, because EPIC only recomputes a small fixed

window of initial tokens at each chunk boundary, it cannot adapt to the distribution shifts that prefix changes induce in the remaining cached context, which in our experiments leads to notable accuracy degradation on reasoning tasks (Table 1).

- **KVCOMM (Ye et al., 2025)**: A retrieval-based framework that approximates cross-context KV states via similarity matching with historical anchors. We use the default configuration from the official repository with an anchor pool size of 20 per agent role. During inference, the system retrieves a subset of nearest anchors (weighted by softmax similarity) to estimate KV offsets. While robust in many scenarios, its rigid matching constraints cause the reuse rate to decline as the number of agents increases.

B.4. Main Evaluation Benchmarks

Tasks and Prompts.

- **GSM8K (Reasoning)**: A dataset of high-quality grade school math problems. Evaluated in a 4-agent chain structure where agents sequentially verify and refine mathematical reasoning steps.
- **HumanEval (Coding)**: A benchmark comprising 164 hand-written Python programming problems. Evaluated in a Developer-Tester loop where the Developer generates code and the Tester provides unit test feedback. The maximum generation length is set to 512 tokens.
- **MMLU (Knowledge)**: A massive multitask benchmark covering 57 subjects across STEM, the humanities, and social sciences. Evaluated using a RAG pipeline (Retriever-Reader-Critic) to test the relay of long contexts containing external knowledge.
- **AIME (Efficiency)**: Used strictly for efficiency scaling laws (Figure 8). We constructed synthetic prompts with controlled input lengths ranging from 512 to 12,288 tokens. The outputs were aligned to ensure fair comparison of TTFT across context lengths.

Hardware and Environment. All experiments were conducted on a server equipped with NVIDIA H20 (96GB) GPUs. The system is implemented on top of PyTorch v2.1.0 and Transformers v4.50.2. We measure TTFT using CUDA Events for precise latency recording.

B.5. Detailed Agent Configurations

In this section, we provide the exact agent roles, system prompts, and interaction topologies employed in our multi-agent experiments.

GSM8K (Reasoning). We utilize a sequential chain structure comprising three distinct agents followed by a decision node. The workflow proceeds as Mathematical Analyst → Inspector → Math Solver → Decision Maker.

- **Agent 1: Mathematical Analyst.**

"You are a mathematical analyst. You will be given a math problem, analysis and code from other agents. You need to first analyze the problem-solving process step by step, where the variables are represented by letters. Then you substitute the values into the analysis process to perform calculations and get the results. The last line of your output contains only the final result without any units, for example: The answer is 140. You will be given some examples you may refer to."

- **Agent 2: Inspector.**

"You are an Inspector. You will be given a math problem, analysis and code from other agents. Check whether the logic/calculation of the problem solving and analysis process is correct(if present). Check whether the code corresponds to the solution analysis(if present). Give your own solving process step by step based on hints. The last line of your output contains only the final result without any units, for example: The answer is 140. You will be given some examples you may refer to."

- **Agent 3: Math Solver.**

"You are a math expert. You will be given a math problem and hints from other agents. Give your own solving process step by step based on hints. The last line of your output contains only the final result without any units, for example: The answer is 140."

- **Decision Node: Decision Maker.**

"You are a decision-maker. You check if the solutions match the question. You prefer simple and direct answers. You will be given a math problem (Q) and solutions from other agents. Read the Question (Q) carefully. It is the absolute truth. If an agent's solution uses the numbers from Q and the logic is simple and correct, ACCEPT it directly. Do NOT add extra steps or change the conditions in Q. Only provide a new solution if the agents made a calculation error. The last line of your output contains only the final result without any units, for example: The answer is 140"

HumanEval (Coding). Our concrete implementation instantiates the conceptual Developer-Tester loop described in the main text as a robust 5-stage pipeline, ensuring rigorous quality control before the final decision aggregation. The execution flow is Project Manager → Algorithm Designer → Programming Expert → Test Analyst → Bug Fixer → Top Decision-Maker.

- **Agent 1: Project Manager.**

"You are a project manager. You will be given a function signature and its docstring by the user. You are responsible for overseeing the overall structure of the code, ensuring that the code is structured to complete the task. Implement code concisely and correctly without pursuing over-engineering. You need to suggest optimal design patterns to ensure that the code follows best practices for maintainability and flexibility. You can specify the overall design of the code, including the classes that need to be defined(maybe none) and the functions used."

- **Agent 2: Algorithm Designer.**

"You are an algorithm designer. You will be given a function signature and its docstring by the user. You need to specify the specific design of the algorithm, including the classes that may be defined and the functions used. You need to generate the detailed documentation, including explanations of the algorithm, usage instructions, and API references. When the implementation logic is complex, you can give the pseudocode logic of the main algorithm. I hope your reply will be more concise. Preferably within fifty words. Don't list too many points."

- **Agent 3: Programming Expert (Developer).**

"You are a programming expert. You will be given a function signature and its docstring by the user. You may be able to get the output results of other agents. They may have passed internal tests, but they may not be completely correct. Write your full implementation (restate the function signature). Use a Python code block to write your response. For example: `print('Hello world!')` Do not include anything other than Python code blocks in your response. Do not change function names and input variable types in tasks."

- **Agent 4: Test Analyst (Tester).**

"You are a test analyst. You will be given a function signature and its docstring by the user. You need to provide problems in the current code or solution based on the test data and possible test feedback in the question. You need to provide additional special use cases, boundary conditions, etc. that should be paid attention to when writing code. You can point out any potential errors in the code."

- **Agent 5: Bug Fixer.**

"You are a bug fixer. You will be given a function signature and its docstring by the user. You need to provide modified and improved python code based on the current overall code. Use a Python code block to write your response."

- **Decision Node: Top Decision-Maker.**

"You are the top decision-maker and are good at analyzing and summarizing other people's opinions, finding errors and giving final answers. And you are an AI that only responds with only python code. You will be given a function signature and its docstring by the user. You may be given the overall code design, algorithm framework, code implementation or test problems. Write your full implementation (restate the function signature). If the prompt given to you contains code that passed internal testing, you can choose the most reliable reply. If there is no code that has passed internal testing in the prompt, you can change it yourself according to the prompt. Use a Python code block to write your response... Do not include anything other than Python code blocks in your response"

MMLU (Knowledge). To mitigate hallucinations, we employ a diverse five-agent group followed by an expert analyzer for final selection. The roles rotate through: Knowledgeable Expert → Wiki Searcher → Critic → Mathematician → Psychologist → Expert Analyzer.

- **Agent 1: Knowledgeable Expert.**

"Please give at most six key entities that need to be searched in wikipedia to solve the problem. Key entities that need to be searched are included between two '@' when output... If there is no entity in the question that needs to be searched in Wikipedia, you don't have to provide it."

- **Agent 2: Wiki Searcher.**

"Please refer to them step by step to give your answer. If the Wikipedia overview is missing or insufficient, you must rely on your internal knowledge to explicitly define the nature, social status, or scientific mechanism of the entities identified by Agent 1 BEFORE choosing an option. Do not guess based on vague associations."

- **Agent 3: Critic.**

"You are an excellent critic. You will review the analysis provided by the Wiki Searcher. Please critique the analysis point by point based on the following criteria: ... 3. Is the final option fully supported by the analysis steps?"

- **Agent 4: Mathematician.**

"You are a mathematician who is good at math games, arithmetic calculation, and long-term planning."

- **Agent 5: Psychologist.**

"You are a psychologist. You are good at psychology, sociology, and philosophy. You give people scientific suggestions that will make them feel better."

- **Decision Node: Expert Analyzer.**

"You are an expert analyzer. You are good at finding the correct option among A, B, C and D based on logic and evidence. You will be given a question with 4 options (A, B, C, D) and analysis from other agents. If the analysis from other agents is logical and correct, use it to support your answer. If the analysis from other agents is wrong, ignore it and use your own reasoning. Do not imitate the other agents' tone; just state the facts. Write a brief analysis. The last line of your output must contain only the option letter, for example: The answer is A"

AIME (Efficiency Evaluation). For the efficiency scaling laws analyzed in Figure 8 and Table 2, we employ a controlled experimental setup to decouple system latency from model generation variance using the AIME 2024 dataset.

- **Dataset Processing:** We construct the task input using a fixed template:

*"You are given a math competition problem.
Problem: {problem}
Task:
- Solve the problem carefully.
- Explain your reasoning step by step.
- Finish by stating ONLY the final answer on the last line."*

To ensure precise control over prefill and decode lengths, the input text is padded with a neutral instruction set to exactly match the target token count:

"Additional instructions:

- Use clear algebra/number theory steps.*
- Check constraints and edge cases.*
- Provide the final answer on the last line."*

- **Agent Configuration:** We utilize a homogeneous agent structure configured to form a strict cumulative context chain. The system prompt is set to empty. Agent i receives a user prompt consisting of the direct concatenation without separators: $\{\text{user_question}\}\{\text{output_agent_0}\} \dots \{\text{output_agent_}(i-1)\}$. This guarantees that the prefill workload for downstream agents scales precisely as defined in the experiment.

C. Discussion

Extension to Heterogeneous Settings. Our current framework focuses on homogeneous multi-agent systems where all agents share identical model architectures. In heterogeneous settings—where agents may use different model families or sizes—direct KV cache transfer becomes infeasible due to mismatched hidden dimensions and attention head configurations. Potential directions include learning lightweight projection layers between architectures or developing distillation-based approaches that transfer semantic information rather than raw KV caches. We leave this exploration to future work.

Hyperparameter Sensitivity. RelayCaching relies on two types of hyperparameters: (1) layer-range boundaries (L_{start} , L_{end} , L_{det}) determined through offline profiling on a calibration dataset, and (2) threshold multipliers (τ_{dev} , τ_{inf}) that are fixed empirical constants applied to runtime-computed statistics (μ_{dev} , μ_{inf}). While both generalize well across our evaluated benchmarks, task distributions with substantially different characteristics may benefit from task-specific calibration or online adaptation.